

# **High-level Digital Design**

## **Lab Assignment 5**

**Implementation of a radix-4 Booth multiplier IP with an AXI4 interface**

## 0. Goals

The goal of this laboratory assignment is the design and implementation of an AXI4 IP based on the 16-bit x 16-bit radix-4 Booth multiplier that was developed in laboratory assignment 4. This IP will be managed by the HPS of the Cyclone V device by means of interrupts. First a guide will be provided in order to construct the IP and the corresponding hardware platform. Afterwards, indications to develop the software application will be provided.

The design will be based on the hardware and software projects that were created for laboratory assignment 2. Therefore, it is suggested to make a backup copy of them by issuing the following commands:

```
cp -r lab1 lab2_hw_backup
```

```
cp -r lab2 lab2_sw_backup
```

## 1. Design of the IP and hardware platform

The multiplier developed in laboratory assignment 4 has to be instantiated in the AXI4 interface provided in the file *axi4\_lite\_booth.vhd*. When making the instantiation you should bear in mind that:

- Input *data\_a* of the multiplier is stored in *slv\_reg0*. It is written by HPS.
- Input *data\_b* of the multiplier is stored in *slv\_reg1*. It is written by HPS.
- Output *result* of the multiplier will be stored in *slv\_reg2*. It is read by HPS, provided by the multiplier.
- Input *start* of the multiplier is stored in *slv\_reg3(0)*. It is written by HPS.
- Output *busy* of the multiplier is mapped to *slv\_reg3(1)*. It is read by HPS, provided by the multiplier.
- Input *ack* of the multiplier is stored in *slv\_reg3(2)*. It is written by HPS.
- If the optional part of laboratory assignment 4 is implemented, then the input *irq\_enable* is stored in *slv\_reg3(3)*. It is written by HPS.
- The output *irq* provided by the multiplier has to be propagated as an external output.

Once the AXI4 interface has been changed you can create the IP following the procedure provided in laboratory assignment 2. When creating this IP, in the *Signals & Interfaces* section you should set the interface for output *irq* as *Interrupt Sender*.

Once the IP has been created you should open the *lab1* project and the system created with platform designer. First of all, you should remove the PIO IP that manage the pushbuttons and the 28-bit counter AXI4 IP. Now you should include the IP corresponding to the Booth multiplier and connect it as it was explained in laboratory assignment 2. You should connect the *irq* output of the Booth multiplier to the *f2h\_irq0* input of the *ARM\_A9\_HPS* component. Then, you should set the value of the IRQ column of this output to 1. In this way, the interrupt generated by the multiplier will be connected to interrupt channel 73, the same as for the PIO IP that manages the switches. This will facilitate the development of the software application based on what was provided for laboratory assignment 2. You should assign to this IP the same base

address that was assigned to the PIO that manages the pushbuttons, 0x0000\_0000. Save the new system and generate the corresponding HDL description.

Before compiling the new system, remember to remove in lab1.vhd the ports and signals corresponding to the pushbuttons and LEDs, since they are no more used in the design. After making these changes, compile the new design and download it to the device.

## 2. Development of the software application

Since the interrupt generated by the IP is connected to the interrupt channel 73 of the HPS, it is possible to use the software code provided for laboratory assignment 2. The configuration of the A9 stack and the GIC are the same, and for the exception handler the only change to be done is the substitution of the call to the function *pushbutton\_ISR()* by a new function, that may be called *booth\_ISR()*.

If you didn't implement the optional part of laboratory assignment 4, in the file lab2.c you should remove the call to the function *config\_KEYS()*, since it is not necessary to enable the interrupt generated by the Booth multiplier. Otherwise, you should change it in order to enable the interrupts generated by the multiplier.

Bearing in mind the base address assigned to the Booth multiplier IP the absolute address of the registers that constitute the AXI4 interface are:

- slv\_reg0: 0xFF200000
- slv\_reg1: 0xFF200004
- slv\_reg2: 0xFF200008
- slv\_reg3: 0xFF20000C

The sequence of actions to be performed by the HPS are:

1. If the optional part of laboratory 4 is implemented, enable the interrupts generated by the multiplier by writing a value 1 in bit 3 of slv\_reg3.
2. Verify that there is not a pending multiplication by checking bit 1 of slv\_reg3 (*busy* output of the multiplier).
3. Clear any pending interrupts by writing a value 1 and then a value 0 on bit 2 of slv\_reg3 (*ack* input of the multiplier).
4. Write the multiplicand on slv\_reg0.
5. Write the multiplier on slv\_reg1.
6. Start the multiplication by writing a value 1 on bit 0 of slv\_reg3 (*start* input of the multiplier).
7. Wait for the multiplication to complete. This can be achieved by checking a flag that is set by the ISR associated with the interrupt generated by the multiplier. Once this is detected the flag should be reset, the *ack* signal activated and then deactivated to reset the *irq* signal and finally the *start* signal should be set to 0.
8. Read the contents of slv\_reg2 (*result* output of the multiplier) and display it on the terminal by means of a call to `printf()` to verify that the result is correct.

Regarding the ISR associated with the multiplier, the function *pushbutton\_ISR()* included in the file *pushbutton\_ISR.c* can be substituted by a function called *booth\_ISR()*. This function should first clear the interrupt by writing a value 1 and then

a value 0 on bit 2 of `slv_reg3` (ack input of the multiplier). Then, a global flag should be set in order to let the application program know that an interrupt coming from the multiplier has been received.